End-to-End Latency Scaling with Context Length on Edge GPU Inference

Motivation: Moving beyond raw compute speed to understand predictability and throughput stability in real-world edge deployments.

Experimental Setup

System Config

- **Model:** GPT-2 Medium (with KV-cache)
- **Environment:** Edge GPU inference engine
- **Method:** Multiple runs per context length bucket

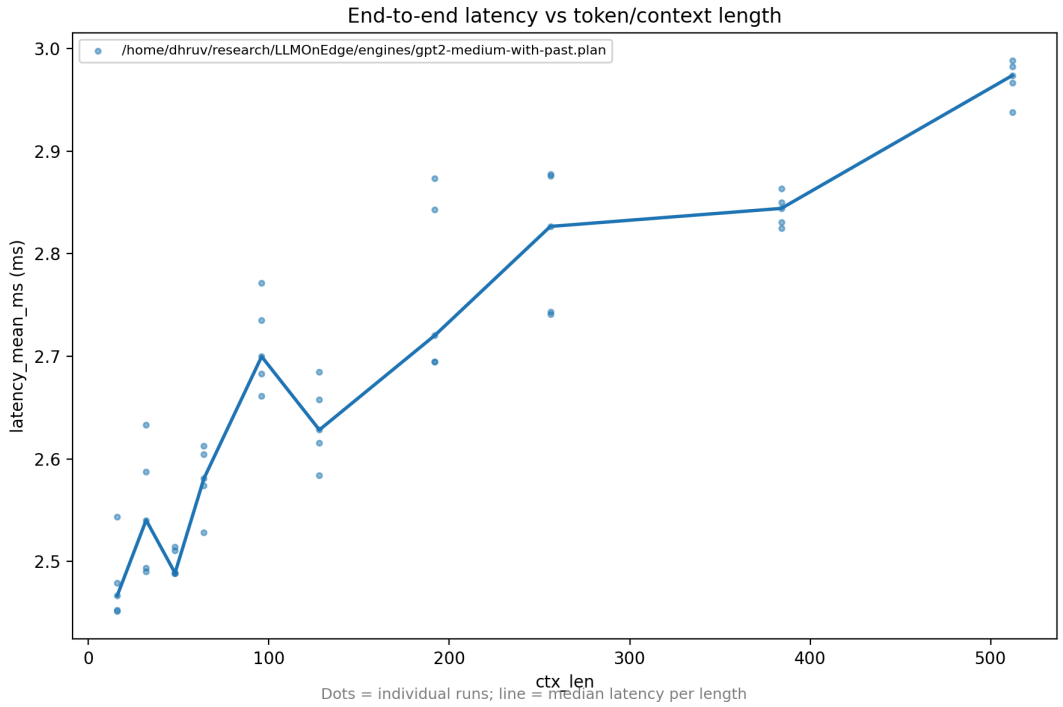
Key Metrics

- End-to-End Latency
- GPU Compute Time
- Transfer + Enqueue Overhead
- Variability (p95, p99, CV)

End-to-End Latency vs Context Length

- Latency increases roughly **linearly** as context length grows.
- Observable small, non-monotonic dips at mid-range context lengths.
- Scatter plot visualization highlights significant **per-run variability**.

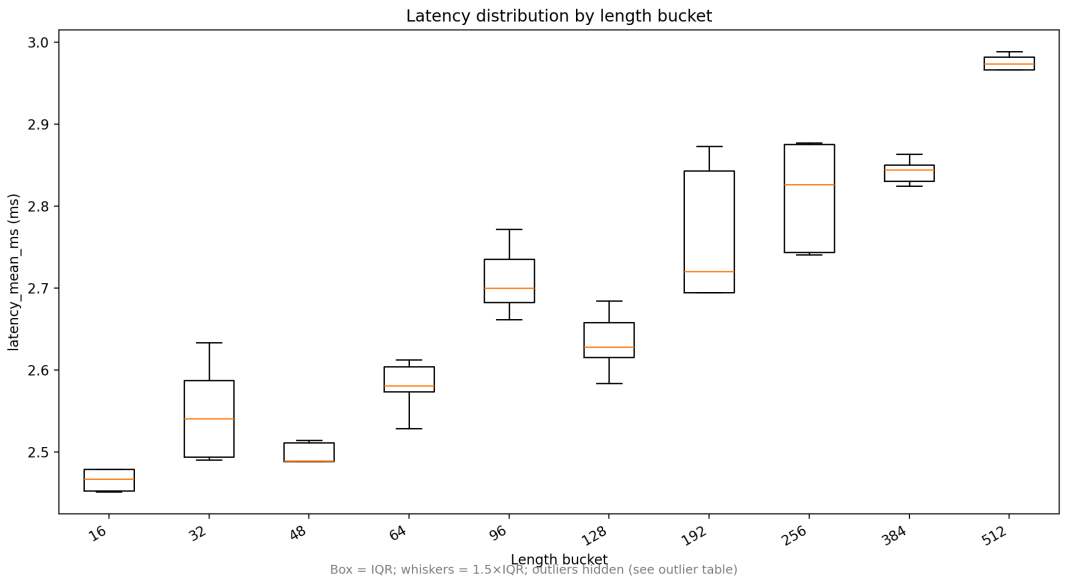
> **Key Takeaway:** Context length is a strong driver of latency, but not the only factor at play.



Latency Distributions by Bucket

- Box plots reveal that the **spread increases** significantly with context length.
- Higher buckets demonstrate wider Interquartile Ranges (IQRs).
- **Outliers** become increasingly frequent and severe at long context lengths.

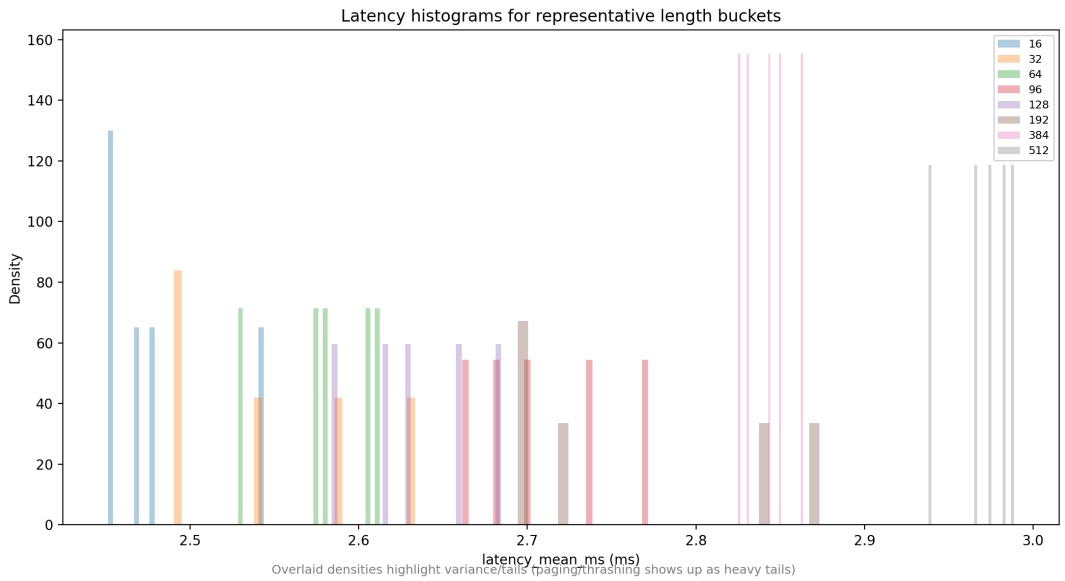
"The median moves smoothly, but the tails are what matter for real systems."



Shape of Latency Distributions

- **Short Contexts:** Tight, near-Gaussian distributions. Predictable performance.
- **Longer Contexts:** Heavy tails emerge.
- Indicates intermittent overhead sources such as memory paging or allocator effects.

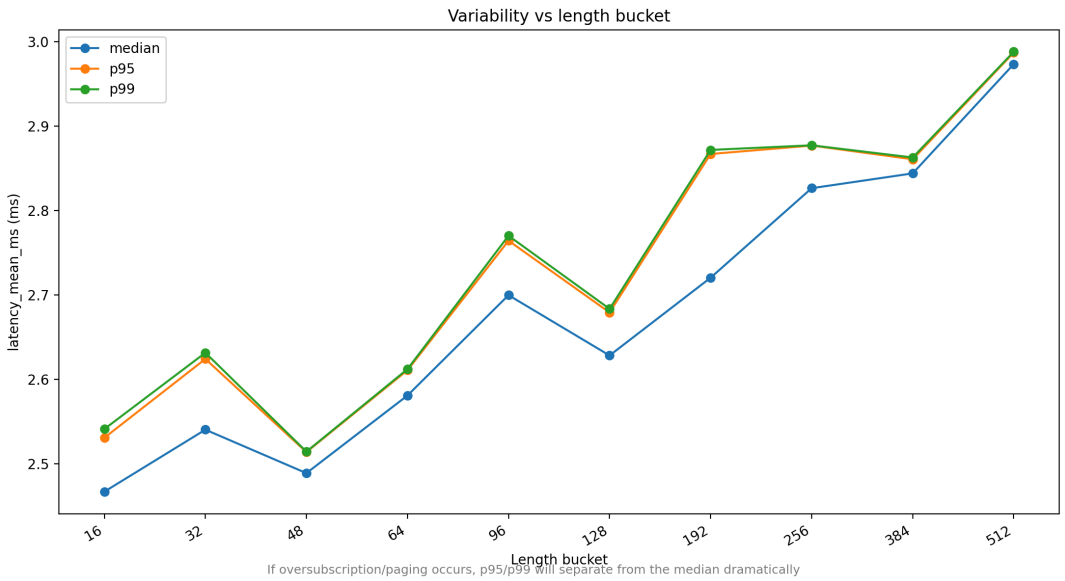
> **Insight:** Latency degradation is not uniform — it becomes probabilistic at scale.



Variability Growth with Context

- **p95 and p99** diverge sharply from the median at higher lengths.
- Variability grows at a faster rate than median latency itself.
- Clear signal of oversubscription or **memory pressure**.

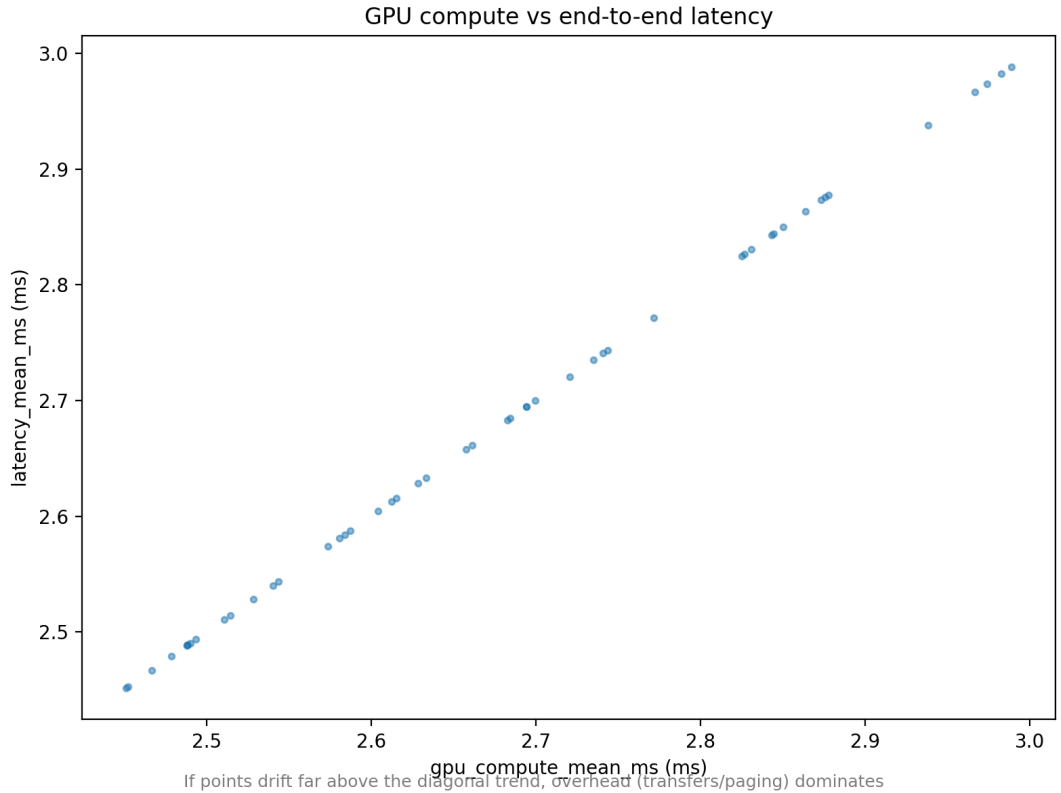
"This is where systems start to feel 'unreliable' even if the average looks fine."



GPU Compute vs End-to-End Latency

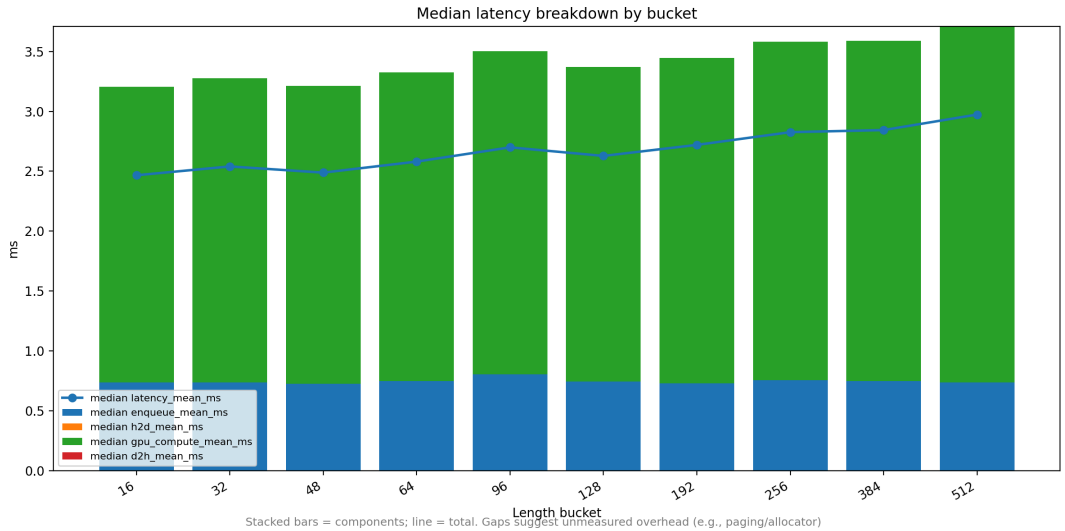
- Strong linear relationship between raw GPU compute and total latency.
- Points **above the diagonal** indicate non-compute overhead.
- This overhead component increases disproportionately with context size.

Conclusion: GPU compute explains most, but not all, of end-to-end latency.



Median Latency Breakdown

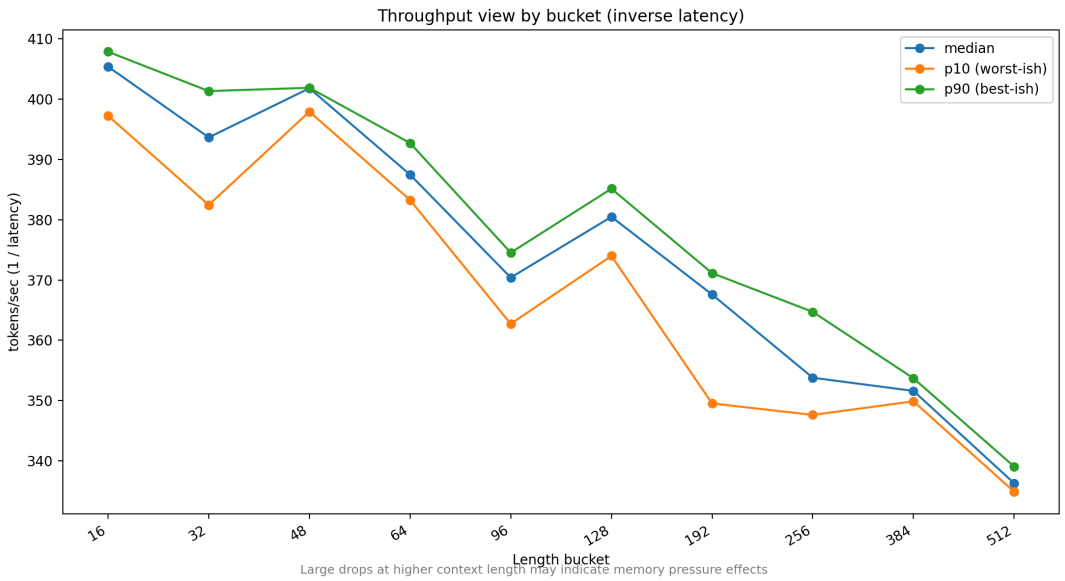
- **GPU Compute** dominates the total latency duration.
- Enqueue and transfer costs remain non-trivial.
- The gap between stacked components and total time reveals **hidden overhead** (e.g., framework overhead, paging).



Throughput (Inverse Latency)

- Throughput declines steadily as context length increases.
- **Worst-case throughput** drops significantly faster than the median.
- Performance "cliffs" appear at high context sizes.

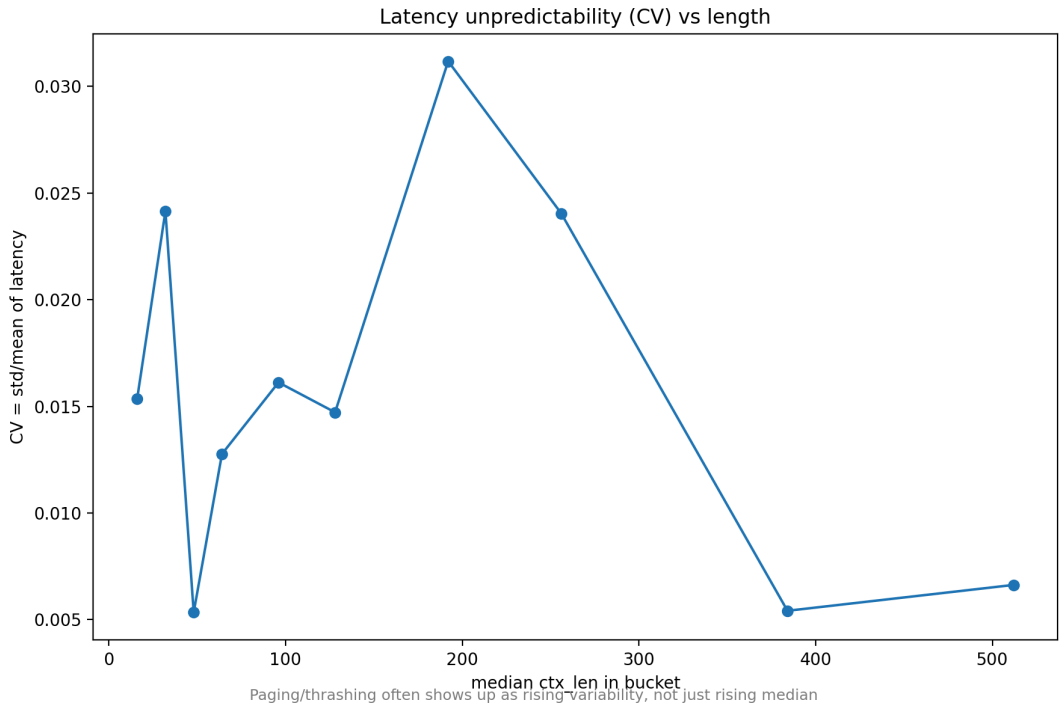
Critical: Long contexts hurt both latency and total system capacity.



Latency Unpredictability (CV)

- Coefficient of Variation (CV) **spikes** at mid-to-high context lengths.
- This increase in variability often **precedes** major increases in median latency.
- Serves as an early warning sign of system stress.

"Rising variability is often a better signal than rising average latency."



Outliers & Failure Modes

Memory Paging

System forced to swap memory to disk, causing massive latency spikes.

KV-Cache Pressure

Growing cache consumes available GPU memory, leading to fragmentation.

Allocator Effects

Overhead from finding contiguous memory blocks for large tensors.

∗Analysis based on outlier clusters at max context length (Refer to Table 09)

Summary of Findings

Scaling: Latency scales linearly with context, but variability scales much faster.

Dominance: GPU compute dominates total time, but system overhead grows disproportionately.

Unpredictability: High context lengths introduce probabilistic latency degradation.

Throughput: System capacity degrades non-linearly near memory limits.

Implications for Edge Deployment

Context Caps

Strictly limiting context length is necessary to improve tail latency guarantees.

Predictability

For real-time edge apps, consistent performance often matters more than peak speed.

Deep Profiling

Performance validation must include p95/p99 metrics, not just averages.

Future Work

Optimization

- Test different allocator strategies to reduce fragmentation.
- Compare advanced batching techniques against single-request inference.

Expansion

- Explicit KV-cache memory profiling to isolate overheads.
- Extend benchmarks to larger models (e.g., Llama-2) and longer context windows.